CS 300 – Chapter 12 Lectures

Sets

Jose de Lima

April 26, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 12 is the *ADT-meta chapter*: it is not an algorithm or a data structure unto itself — it is the survey of *the set abstract data type*, the operations that make sense on top of it, the impls cs-300 has already given you elsewhere, and the engineering rules for picking between them. ch. 5 covered hash tables ($O(1)$ expected membership, no order). ch. 9 covered AVL and red-black trees ($O(\log n)$ ordered membership). ch. 11 covered B-trees and B+ trees ($O(\log_K n)$ disk-friendly ordered membership). Each of those is one way to implement a Set. This chapter is the meta-layer: *given the Set ADT, how do you choose, and where do the corner cases (multiset, bitset, partition, probabilistic) live?*

What you add to your toolkit:

- **The Set ADT itself.** The three core ops (`contains` / `insert` / `erase`) plus the structural ops (`union` / `intersection` / `difference` / `symmetric difference` / `subset` / `powerset`).
- **The impl decision matrix.** Which Set implementation to pick given (a) ordered vs unordered iteration; (b) point-query vs range-query workload; (c) static vs dynamic; (d) memory budget; (e) duplicate-key allowance; (f) approximate-membership tolerance.
- **Multiset.** Duplicates allowed; same impl machinery with $\leq$ replacing $<$ in comparisons. `std::multiset` is RB-backed with three well-defined behavioural deltas from `std::set`.
- **Bitset / characteristic vector.** When the key universe is small and dense, the direct-access array form of a Set — one bit per possible key, $O(1)$ point-query, $O(n/w)$ word-parallel set ops.
- **Disjoint-Set Union (Union-Find).** The set-of-disjoint-sets ADT that powers Kruskal's MST (ch. 10 §10.12). Parent-pointer forest + union-by-rank + path-compression $\Rightarrow O(\alpha(n))$ amortised — effectively constant.
- **Bloom filter.** Probabilistic Set membership — one-sided error (false positives possible, no false negatives). Real production tool: RocksDB, PostgreSQL extensions, CDN edges, DNS negative caches.
- **Perfect hashing.** The FKS two-level construction for static sets where you want $O(1)$ *worst-case* (not expected) lookup.

Mastery checklist — you should be able to, without references:

1. Pick the right Set impl at first sight: ordered iteration (RB / B-tree), unordered $O(1)$ lookup (hash), small dense universe (bitset), append-only-with-membership-queries (Bloom filter), partition (Union-Find), static / sealed (perfect hash). Justify each in one sentence.
2. Implement set union / intersection / difference on (a) two `std::sets` by sorted-merge ($O(|A| + |B|)$) and (b) two `std::unordered_sets` by hash-probe ($O(|A| \cdot \text{contains}_B)$ worst case, typically

$O(|A| + |B|)$ expected). Explain the algorithmic difference.

3. State the Multiset distinction (duplicates allowed) and identify which set ops change behaviour: union (sum-multiplicities or max-multiplicities — both are defensible; pick one and stick), intersection (min-multiplicities), difference ($\max(0, m_A(x) - m_B(x))$).
4. Implement a bitset as `std::vector<uint64_t>` + bit-twiddling indexing; argue why it gives $O(1)$ point-query, $O(n/w)$ iteration, and $O(n)$ space where n is the universe size.
5. State the Disjoint-Set Union invariants, sketch the path-compression + union-by-rank optimisation, and state the inverse-Ackermann amortised bound: $O(\alpha(n))$ per operation, with $\alpha(n) \leq 4$ for any $n \leq 2^{65536}$.
6. State the Bloom-filter false-positive-rate formula $\left(1 - e^{-kn/m}\right)^k$ for k hash functions, n elements, m bits, derive optimal $k \approx (m/n) \ln 2$, and identify three production use cases (network filters, DB existence pre-check, LSM compaction).
7. Identify which `std::` containers are sets vs maps vs multisets and the underlying data structure of each (RB / hash / etc.). Distinguish "associative" (ordered) from "unordered associative" in the C++ STL nomenclature.

Looking ahead. Past the cs-300 boundary the Set ADT keeps extending: concurrent / lock-free hash sets (Cliff Click's non-blocking hash map, Java's `ConcurrentSkipListSet`, RCU read-mostly sets), set theory proper (powerset, partial orders, lattices, Boolean algebras), and the sparse-bitmap research line (roaring bitmaps, used in PostgreSQL / Druid / Lucene). The production-references notebox in §12.8 is the entry point.

Key idea

The set ADT in one sentence. A set is a collection of distinct elements with fast membership queries. It's the data structure you reach for any time you care about "have I seen this before" or "how do these two collections compare," and almost nothing else. Everything in this chapter is about the operations that make sense on top of that single primitive.

1 12.1 The Set ADT

What a set is

Definition —

Set. An unordered collection of distinct elements. No duplicates, no position. $\{3, 7, 9\} = \{9, 7, 3\}$.

The three things a set gives you:

- `add(x)`: insert x if not already present; no-op otherwise.
- `remove(x)`: remove x if present.
- `contains(x)`: membership query.

That's it. Everything else (union, intersection, filter, map, subset) is built on top of these three.

Keys vs. elements

When the elements are primitives (ints, strings), each element is its own key. When the elements are objects – a `Student` with `name`, `phone`, `id` – the set uses one field as the key and compares elements by that. The rest of the object is just cargo.

```
// C++ lets you choose: the element type IS the key, or you supply
// a hash/comparator that extracts a key.
std::unordered_set<int> primes;
std::unordered_set<Student, StudentHash, StudentEq> roster;

// Or use a map if you need key/value separation.
std::unordered_map<int, Student> rosterById;
```

Aside

In Python, sets use `__hash__` and `__eq__`; in Java, it's `hashCode()` and `equals()`. The contract is always the same: equal elements must hash equal. If you break that contract, the set is not a set – it's a corrupt data structure waiting to lose an element.

Where the Set impls live in cs-300

The Set ADT has no canonical implementation — the right impl depends on the workload. cs-300 has already given you four of them:

Aside Forward references to existing Set impls

- **ch. 5 — Hash tables.** Open-addressing and chaining; `std::unordered_set` sits on top of this. $O(1)$ expected `contains` / `insert` / `erase`. No order. The default Set impl when you do not need ordered iteration.
- **ch. 6 — Binary search trees (unbalanced).** The baseline ordered Set impl. $O(\log n)$ expected on random inputs, $O(n)$ worst case — adversarial inputs degenerate into a linked list. Pedagogical, not production.
- **ch. 9 — AVL and red-black trees.** Balanced binary search trees. $O(\log n)$ *worst-case* on all three core ops. `std::set` / `std::map` (and Java `TreeSet` / `TreeMap`) are red-black under the hood. The default Set impl when you *do* need ordered iteration or `lower_bound`.
- **ch. 11 — B-trees / B+ trees.** Shallow-and-wide balanced search trees. $O(\log_K n)$ on all three core ops, where the fanout K is tuned to the memory hierarchy block size. The Set impl on disk: SQLite, PostgreSQL `btree`, MySQL InnoDB, ext4 htree.

This chapter does not re-derive any of those. It treats them as black-box impls of the same ADT and asks: *when do you pick which?* Plus the impls cs-300 has *not* covered yet — bitset (§12.5), DSU (§12.6), Bloom filter (§12.7) — which fill out the Set-impl space.

Set vs. Sequence: the two ADTs CS 300 covers

Aside Set ADT vs. Sequence ADT

The two big-picture ADTs cs-300 unfolds are *Sequence* and *Set*.

- **Sequence ADT** (ch. 1–ch. 4): the "get-by-index" interface. Order matters. Position is a first-class query (`at`, `front`, `back`, `insert_at`). Implementations: dynamic array (ch. 3), linked list (ch. 4), `std::vector` / `std::deque` / `std::list`.
- **Set ADT** (this chapter): the "contains-key" interface. Order is incidental (or absent). Position is not a query at all. Implementations: hash (ch. 5), BST (ch. 6), balanced BST (ch. 9), B-tree (ch. 11), bitset (§12.5), DSU (§12.6), Bloom (§12.7).

The OCW 6.006 framing (Recitation r01) is sharp: *a set is just a sequence with the contains-key operation prioritised.* Every Set impl is somewhere on the spectrum between "store the keys contiguously and search" (sorted array) and "hash the keys into buckets and probe." Once you internalise this, the chapter's impl decision matrix collapses to "which axis of the trade-off do I care about today?"

Subset testing

$X \subseteq Y$ iff every element of X is in Y. Trivial algorithm:

```
bool isSubset(const std::unordered_set<int>& sub,
              const std::unordered_set<int>& sup) {
    for (int x : sub)
        if (!sup.count(x)) return false;
    return true;
}
```

Cost: $O(|X|)$ if `contains` is $O(1)$ (hash set). If `sup` is a sorted structure (tree set), it's $O(|X| \log |Y|)$.

2 12.2 Set Operations

Union, intersection, difference

Definition —

$X \cup Y$: everything in either set. $X \cap Y$: everything in both sets. $X \setminus Y$: everything in X but not in Y.

Union and intersection are commutative ($X \cup Y = Y \cup X$). Difference is not.

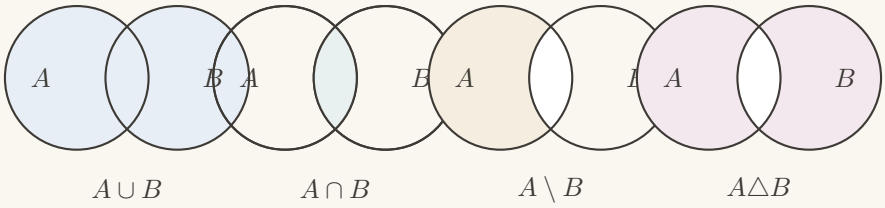
Example

$\{54, 19, 75\} \cup \{75, 12\} = \{12, 19, 54, 75\}$
$\{54, 19, 75\} \cap \{75, 12\} = \{75\}$
$\{54, 19, 75\} \setminus \{75, 12\} = \{19, 54\}$
$\{75, 12\} \setminus \{54, 19, 75\} = \{12\}$

The picture: union / intersection / difference / symmetric difference

The four most common binary set operations have a clean Venn-diagram visual. $A \triangle B$ (*symmetric difference*) is the XOR — elements in exactly one of the two sets:

$$A \triangle B \;=\; (A \cup B) \setminus (A \cap B) \;=\; (A \setminus B) \cup (B \setminus A).$$



The fifth operation, *subset testing* ($A \subseteq B$), is not a Venn region — it is a yes/no predicate ("is A's circle entirely inside B's circle?"). Subsection below.

C++ implementations

The `<algorithm>` header provides set operations on *sorted ranges*, which means any sorted container:

```
#include <set>
#include <algorithm>
#include <iterator>

std::set<int> a{54, 19, 75}, b{75, 12}, out;

std::set_union(a.begin(), a.end(), b.begin(), b.end(),
               std::inserter(out, out.end()));
// out = {12, 19, 54, 75}

out.clear();
std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
                      std::inserter(out, out.end()));
// out = {75}

out.clear();
std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
                    std::inserter(out, out.end()));
// out = {19, 54}
```

For `std::unordered_set`, you write the loops explicitly:

```
std::unordered_set<int> setUnion(const std::unordered_set<int>& a,
                                 const std::unordered_set<int>& b) {
    std::unordered_set<int> r(a);
    r.insert(b.begin(), b.end());
    return r;
}

std::unordered_set<int> setIntersect(const std::unordered_set<int>& a,
                                     const std::unordered_set<int>& b) {
    // Iterate the smaller one -- saves time.
    const auto& smaller = a.size() <= b.size() ? a : b;
```

```
    const auto& larger  = a.size() <= b.size() ? b : a;
    std::unordered_set<int> r;
    for (int x : smaller)
        if (larger.count(x)) r.insert(x);
    return r;
}

std::unordered_set<int> setDiff(const std::unordered_set<int>& a,
                                const std::unordered_set<int>& b) {
    std::unordered_set<int> r;
    for (int x : a)
        if (!b.count(x)) r.insert(x);
    return r;
}
```

Gotcha

Always iterate the smaller set. For intersection, this shaves the cost from $O(|A|)$ to $O(\min(|A|, |B|))$ – a huge win when one side is tiny. It's the single most common optimization in real set-heavy code.

Cost analysis

Assume hash-based sets with $O(1)$ expected insert and lookup.

Operation	Cost	Why
Union $\lvert A\rvert + \lvert B\rvert$	$O(\lvert A\rvert + \lvert B\rvert)$	insert all
Intersection	$O(\min(\lvert A\rvert, \lvert B\rvert))$	scan smaller, probe larger
Difference $A \setminus B$	$O(\lvert A\rvert)$	scan A, probe B
Subset $A \subseteq B$	$O(\lvert A\rvert)$	scan A, probe B

For sorted (tree) sets, add a $\log n$ factor for each membership probe.

Key idea

Sorted vs. hashed, revisited. With sorted sets, the two-pointer merge algorithm computes union/intersection/difference in $O(|A| + |B|)$, matching the hashed version without any per-element hash. On small sets that fit in cache, the sorted version can actually be *faster*, because hashing has nontrivial constant overhead. For large sets with large element types, hashed wins. Benchmark before picking.

Definition —

Algorithmic difference: sorted-merge vs. hash-probe. The two impl families compute the same set-theoretic answer via fundamentally different algorithms:

- **Sorted-merge** (any sorted-range impl — `std::set`, `std::vector` after `sort`, B-tree leaf scan): two pointers walk A and B in lock-step. Each comparison advances at least one pointer. Total work: $\Theta(|A| + |B|)$ *deterministic worst case*, no hashing required.
- **Hash-probe** (`std::unordered_set` + `count` loop): for each element of one set, a single hash + bucket-probe against the other. $O(|A| + |B|)$ *expected* (assuming a well-distributed hash), but $O(|A| \cdot |B|)$ worst case under adversarial hash collisions.

The two are asymptotically equivalent in the average case. They diverge on (a) constants (hashing

eats $\sim$ 50–200 cycles per op even when the bucket lookup is $O(1)$; merge does pure comparisons, no hash), (b) cache locality (sorted-merge streams two arrays linearly; hash-probe touches random bucket addresses), and (c) worst-case guarantees (sorted-merge is deterministic; hash is amortised + assumes good hash).

Example Sorted-merge vs. hash-probe, worked

$A = \{1, 3, 5, 7\}$, $B = \{3, 4, 5, 6\}$.

Sorted-merge $A \cup B$ on `std::set`:

- Initialise pointers $i, j = 0, 0$. Output $\{\}$.
- Compare $A[0] = 1, B[0] = 3$: $1 < 3$, emit 1, $i \to 1$. Output $\{1\}$.
- Compare $A[1] = 3, B[0] = 3$: equal, emit 3, advance both. Output $\{1, 3\}$.
- Compare $A[2] = 5, B[1] = 4$: $5 > 4$, emit 4, $j \to 2$. Output $\{1, 3, 4\}$.
- Compare $A[2] = 5, B[2] = 5$: equal, emit 5, advance both. Output $\{1, 3, 4, 5\}$.
- Compare $A[3] = 7, B[3] = 6$: $7 > 6$, emit 6, $j \to 4$ (exhausted). Output $\{1, 3, 4, 5, 6\}$.
- Drain A: emit 7. Output $\{1, 3, 4, 5, 6, 7\}$.
- *Cost*: 5 comparisons + 6 emits. $\Theta(|A| + |B|) = \Theta(8)$ work; zero hashes.

Hash-probe $A \cup B$ on `std::unordered_set`:

- Copy A into output: 4 hashes + 4 inserts. Output $\{1, 3, 5, 7\}$.
- For each $b \in B$: hash b, probe output, insert if absent.
  - $b = 3$: hash, probe finds it, no insert.
  - $b = 4$: hash, probe miss, insert.
  - $b = 5$: hash, probe finds it, no insert.
  - $b = 6$: hash, probe miss, insert.

  Output $\{1, 3, 4, 5, 6, 7\}$ (*unordered*).
- *Cost*: 8 hashes + 6 bucket probes + 6 inserts. $O(|A| + |B|)$ expected; constant factor $\sim 3\times$ higher than sorted-merge for small int keys due to hash cost.

Both produce the same set; sorted-merge produces *sorted output for free*, hash-probe gives no order guarantee.

Symmetric difference

Definition —

Symmetric difference. $A \triangle B$ is the set of elements in *exactly one* of A and B:

$$A \triangle B \;=\; (A \setminus B) \cup (B \setminus A) \;=\; (A \cup B) \setminus (A \cap B).$$

Commutative ($A \triangle B = B \triangle A$); not associative beyond the standard set-theoretic identity.

The `<algorithm>` header provides this directly on sorted ranges:

```
#include <set>
#include <algorithm>
#include <iterator>
```

```
std::set<int> a{1, 3, 5, 7}, b{3, 4, 5, 6}, out;
std::set_symmetric_difference(
    a.begin(), a.end(), b.begin(), b.end(),
    std::inserter(out, out.end()));
// out = {1, 4, 6, 7}
```

For unordered sets, the hand-rolled form is `setUnion(setDiff(a, b), setDiff(b, a))` or, equivalently, $O(|A| + |B|)$ via two scan-and-probe passes. Use cases: change detection between two snapshots (A = "yesterday's user IDs," B = "today's"; $A \triangle B$ = churn), file diffs, version-control "these lines are in exactly one of the two revisions."

Subset testing, formally

The `isSubset` algorithm sketched in §12.1 is the canonical form. Cost analysis warrants a separate look:

- **Hashed superset:** $O(|A|)$ expected, since each $\texttt{contains}_B(a)$ is $O(1)$ expected.
- **Sorted (tree / B-tree) superset:** $O(|A| \cdot \log |B|)$ deterministic — one tree probe per element of A.
- **Both sides sorted:** fall back to a sorted-merge variant. Two pointers, advance the smaller; if at any point you advance off the end of A before exhausting it via matches, $A \not\subseteq B$. Cost $O(|A| + |B|)$ — *tighter than* $|A| \log |B|$ *when* $|A|$ *is close to* $|B|$, looser when $|A| \ll |B|$.

The pick: hashed superset for small $|A|$ relative to $|B|$ ("does this 5-element query set fit inside this million-element cache?"); sorted-merge for $|A| \approx |B|$ ("are these two billion-row tables equal?").

Strict subset ($A \subseteq B$ *and* $A \neq B$) is the same algorithm plus a $|A| < |B|$ check.

Power set

Definition —

Power set. $\mathcal{P}(X) = \{S : S \subseteq X\}$ — the set of all subsets of X, including $\emptyset$ and X itself. $|\mathcal{P}(X)| = 2^{|X|}$.

Brute enumeration: each subset corresponds to a length-n bitmask where bit i indicates whether $X[i]$ is included. C++:

```
#include <vector>
#include <cstdint>

template<typename T>
std::vector<std::vector<T>> powerSet(const std::vector<T>& xs) {
    const std::size_t n = xs.size();
    std::vector<std::vector<T>> result;
    result.reserve(1ULL << n);                // 2^n
    for (std::uint64_t mask = 0; mask < (1ULL << n); ++mask) {
        std::vector<T> sub;
        for (std::size_t i = 0; i < n; ++i)
            if (mask & (1ULL << i)) sub.push_back(xs[i]);
        result.push_back(std::move(sub));
    }
```

```
    return result;
}
```

Cost: $\Theta(n \cdot 2^n)$ time and space. Practical only for $n \leq 20$-ish ($2^{20} \approx 10^6$); past that, you do not *enumerate* the power set, you *iterate* subsets lazily (same loop, no materialised `result`).

Aside

Why this lives in the bitset corner of the chapter. The `mask` variable above is a single-word bitset over the universe $\{0, 1, \ldots, n-1\}$. Power-set enumeration is the canonical use case for the bitset representation in §12.5: when the universe is small enough to fit in a 64-bit word, every subset *is* a single `uint64_t`. Set ops on the subsets become bitwise ops on the words.

Filter and map

Two higher-order operations borrowed from functional programming:

Definition —

Filter. Given a predicate p, return the subset of elements for which p is true.
Map. Given a function f, return the set $\{f(x) : x \in X\}$. The result may be smaller than the input if f is not injective (multiple inputs can map to the same output).

```
#include <unordered_set>
#include <functional>

template<typename T, typename Pred>
std::unordered_set<T> setFilter(const std::unordered_set<T>& s, Pred p) {
    std::unordered_set<T> r;
    for (const auto& x : s) if (p(x)) r.insert(x);
    return r;
}

template<typename T, typename U, typename Fn>
std::unordered_set<U> setMap(const std::unordered_set<T>& s, Fn f) {
    std::unordered_set<U> r;
    for (const auto& x : s) r.insert(f(x));
    return r;
}

// Usage:
auto evens = setFilter(nums, [](int x){ return x % 2 == 0; });
auto lengths = setMap<std::string, size_t>(words,
                    [](const std::string& w){ return w.size(); });
```

Python / JavaScript / Rust / Haskell / Scala all have these as first-class methods on their set (and collection) types. C++ got closer in C++20 with `std::ranges::filter_view` and `std::ranges::transform_view`, but the syntax is still clunky compared to `xs.filter(...).map(...)` in better-styled languages.

Aside

Filter is a subset; map is not always. Filter always returns a subset of the original. Map can shrink the set if collisions occur under $f\colon \{1, -1\} \to \{1\}$ via $f(x) = x^2$. If you need one-to-one semantics, use a list or multiset and track duplicates explicitly.

3 12.3 Static vs. Dynamic Sets

Definition —

Dynamic set. Can be modified after construction – supports add and remove. **Static set.** Fixed at construction time. Supports read-only and new-set-producing operations (union, intersection, filter, …), but not in-place add or remove.

Operation	Dynamic	Static
Construct from collection	yes	yes
Count elements	yes	yes
Search / contains	yes	yes
Add	yes	no
Remove	yes	no
Union / intersection / difference (return new)	yes	yes
Filter / map (return new)	yes	yes

Why would you ever want a static set?

Key idea

A static set is *immutable*. That buys you several things a mutable set can't:

- **Thread safety for free.** If nothing can change, no locks are needed.
- **Aggressive optimization.** The implementation can use a perfect hash (no collisions), a sorted array (cache-friendly, no per-bucket pointers), or compile-time constants.
- **Safe sharing.** Hand the same set to a dozen consumers and no one can break it.
- **Hashability.** An immutable set can itself be a key in another set or map. (Python's `frozenset` exists for exactly this.)

Choosing in practice

Use case	Choice
Allowed-country codes loaded at startup	Static – set once, read forever
Stop-words for a search engine	Static – fixed per language
Users currently online	Dynamic – changes constantly
Session IDs issued today	Dynamic – grows over time
Finalized vote tallies (after the election)	Static – sealed after a deadline
Shopping cart contents	Dynamic – user modifies it

C++ idioms for static sets

C++ doesn't have a built-in frozen set. Common workarounds:

```
// 1. Wrap in const.
const std::unordered_set<std::string> STOP_WORDS{"a", "an", "the", "of"};

// 2. C++17 initializer at static scope.
constexpr std::array<int, 4> ALLOWED_CODES{200, 301, 302, 404};
// Binary-search in a sorted array -- cache-friendly, no dynamic allocation.

// 3. Compile-time perfect hash via gperf, for pathological performance.
```

Perfect hashing for static sets

Definition —

Perfect hash function. A hash function $h : U \to \{0, \ldots, m-1\}$ that is *injective on a fixed key set* $S \subseteq U$: distinct $x, y \in S$ map to distinct slots. No collisions, ever, for any $x \in S$. This gives $O(1)$ *worst-case* (not just expected) lookup.

Perfect hashing requires the key set to be known up front (*the set must be static*); given that, you can construct a hash function custom-tailored to those keys.

The canonical construction is **FKS** (Fredman, Komlós, and Szemerédi, 1984) — a two-level scheme. CLRS §11.5 walks the proof; the architecture in two sentences:

1. **Level 1.** Hash all n keys into a primary table of size $m_1 = n$ slots using a universal hash h_1. Some slots will collide — expected.
2. **Level 2.** For each level-1 slot containing k keys, allocate a secondary hash table of size $m_2 = k^2$ and pick a level-2 hash h_2 (drawn from the same universal family) such that all k keys land in distinct level-2 slots. The k^2 size ensures a perfect h_2 exists with probability $\geq 1/2$ — a few rejection-sampling rounds find one.

Total expected space: $O(n)$ (the $\sum k_i^2$ telescopes to $O(n)$ under universal hashing). Lookup: two hash evaluations + two slot reads. **Strictly $O(1)$ worst case for any key in S.**

In production: `gperf` (GNU Perfect Hash Function Generator) takes a list of strings at build time and emits a C/C++ hash function and lookup table. Used for keyword sets in compilers, HTTP-header tables, MIME types, anywhere the universe is small, fixed, and lookup speed matters. Modern alternatives include `cmph` (C minimal perfect hashing library) and `phf` (Rust's compile-time perfect-hash crate).

When to reach for it: static set, small enough that construction time is not a barrier ($\leq 10^5$ keys is comfortable; 10^6 is feasible), and the marginal cost of expected-case-only lookup matters — compiler keyword tables, kernel syscall tables, embedded firmware look-up.

Aside Sealed / frozen sets in modern languages

The "the set will never change after construction" contract is expressed structurally in most languages other than C++:

- **Python:** `frozenset({1, 2, 3})` is a first-class type; mutation methods (`add`, `discard`) do not exist on it. Hashable — a `frozenset` can be a key in another set or dict.
- **Java:** `Set.of(1, 2, 3)` (Java 9+) returns an immutable `Set<Integer>`; mutation throws

UnsupportedOperationException. Older Java code uses Collections.unmodifiableSet(...).

- **Rust:** idiom is BTreeSet or HashSet bound through an immutable reference (&BTreeSet<T>); the borrow checker enforces no-mutation statically. The phf crate gives true compile-time perfect-hash sets with zero runtime construction cost.
- **Haskell / OCaml:** all collections are immutable by default; "frozen" is the only flavour.

The C++ workarounds in the listing above approximate this structural guarantee through convention — const-wrapping and constexpr std::array are syntactic shields, not structural ones. The next subsection's notebox elaborates.

Aside

Immutability is a spectrum. "Const" in C++ is syntactic – someone can still const_cast it away. Python's frozenset is structurally immutable. Rust and Haskell make the distinction first-class. If your codebase has a convention that "const means truly immutable," lean into static sets; if not, you're one const_cast from heisenbugs.

Sets you'll actually use

Container	Order	Typical implementation
std::set	sorted	red-black tree (ch. 9)
std::unordered_set	hash order	open-addressed hash table
std::multiset	sorted, allows dups	red-black tree
Python set	hash order	open-addressed hash table
Python frozenset	hash order	immutable hash table
Java HashSet / TreeSet	–	hash / red-black
Rust HashSet / BTreeSet	–	hash / B-tree

4 12.4 Multiset (duplicates allowed)

Definition —

Multiset / bag. A collection like a Set, but each element carries a *multiplicity* $m(x) \geq 0$ (a non-negative integer) — the number of copies. A standard set is the multiset where $m(x) \in \{0, 1\}$ for all x. Order is still incidental; what changes is that $\{1, 1, 3\}$ and $\{1, 3\}$ are different multisets (different multiplicity for 1).

The terms "multiset" and "bag" are interchangeable in algorithm-textbook usage; "multiset" is the C++ STL spelling (std::multiset / std::multimap).

The STL multiset, three behavioural deltas

std::multiset<K> is RB-backed (same as std::set), ordered, allows duplicates. The three operations whose behaviour *changes* versus std::set:

- **count(k) returns 0 or more** (vs. 0 or 1 on std::set). Cost $O(\log n + m(k))$ — one tree probe to find the first occurrence, then a linear scan of the duplicates run.
- **erase(k) removes ALL occurrences** of k (vs. at most one on std::set). To remove a single occurrence,

take an iterator from `find(k)` and `erase(it)`.

- **`equal_range(k)` returns** [**`lower_bound`**, **`upper_bound`**) — the half-open range of all occurrences of k. The canonical way to iterate all duplicates without re-probing.

```
#include <set>
#include <iostream>

std::multiset<int> ms{1, 3, 3, 3, 5, 7};

std::cout << ms.count(3);              // 3
ms.erase(3);                           // removes ALL three 3s
// ms is now {1, 5, 7}

ms.insert({2, 2, 2, 4});
auto [lo, hi] = ms.equal_range(2);
for (auto it = lo; it != hi; ++it)
    std::cout << *it << ' ';           // 2 2 2

// Remove exactly one occurrence of 2.
ms.erase(ms.find(2));
```

Set ops on multisets, contrasted

The set-theoretic operations generalise to multisets, but *multiplicities have to be combined*, and the choice of combiner is the only thing the standard does not legislate.

Op	Set rule	Multiset rule (multiplicities)
$A \cup B$	$x \in A \vee x \in B$	$m_A(x) + m_B(x)$ *(sum, default)*
$A \cup B$ (alt)	—	$\max(m_A(x), m_B(x))$ *(max, also valid)*
$A \cap B$	$x \in A \wedge x \in B$	$\min(m_A(x), m_B(x))$
$A \setminus B$	$x \in A \wedge x \notin B$	$\max(0, m_A(x) - m_B(x))$
$A \triangle B$	$(A \setminus B) \cup (B \setminus A)$	$\lvert m_A(x) - m_B(x)\rvert$
$A \subseteq B$	$\forall x : x \in A \Rightarrow x \in B$	$\forall x : m_A(x) \leq m_B(x)$

Aside Multiset union: sum vs. max

The literature does not pick: CLRS does not legislate, the C++ standard's `std::set_union` on `std::multiset` ranges implements neither in full — it produces the *max-multiplicities* result (one copy per distinct duplicate between the two ranges). `std::multiset::merge`, by contrast, gives *sum-multiplicities* (every element from both sources ends up in the destination).

This chapter's default: sum-multiplicities. It matches `merge`, matches the algebraic identity $|A \cup B| = |A| + |B| - |A \cap B|$ (which only holds with sum + min on multisets), and is what production code in counting / inventory / histogram contexts actually means by "union." Document whichever convention you pick at the call site — the convention is the contract.

Cross-link: heaps as multisets

ch. 7 (heaps) already works on multiset semantics: a priority queue with two equal-priority elements stores both. The heap-property comparison uses $\leq$ (or $\geq$ for max-heaps), not $<$ — this is the multiset-friendly form, and it lets duplicate priorities coexist in the heap without pseudo-tiebreaking. Almost every concrete

impl in cs-300 (heap, BST, hash table) extends to multisets by relaxing one strict-inequality to non-strict. ch. 7's gap-analysis Set/Multiset note made this explicit; §12.4 is the ADT-side framing.

5 12.5 Bitset / characteristic vector

Definition —

Bitset (characteristic vector). A direct-access array representation of a Set over a small dense universe $U = \{0, 1, \ldots, n-1\}$: an n-bit array where bit i is 1 iff $i \in S$. Membership is a single bit read; insert / erase are single bit writes. $O(1)$ point-query. $O(n)$ space (where n is the universe size, *not* the set cardinality).

Aside When a bitset is the right Set impl

Bitsets dominate when the key universe is *small and dense*:

- **ASCII characters:** 128 bits = 16 bytes. The "set of allowed delimiters" for a parser.
- **Vertex IDs in a graph with $|V| = 1000$:** 1000 bits $= 125$ bytes per visited-set. ch. 10's BFS / DFS visited-set is often a bitset in production.
- **Days of the year a job ran:** 365 bits = 46 bytes.
- **Compiler flag set, syscall flag set, CPU feature flags:** dozens to hundreds of bits.

The flip side: when the universe is *sparse* ("set of 64-bit integer IDs, in use $\sim 1\%$ of the space"), a naive bitset is 2^{64} bits = 2 exabytes of zeros. Use a hash set or a roaring bitmap (compressed bitset) instead.

Implementations

C++ gives three flavours, picked by whether n is known at compile time:

- **`std::bitset<N>`** — N a compile-time constant. All ops are `constexpr`-eligible; the compiler often unrolls the word-loop. Fixed size; cannot grow or shrink at runtime. Standard-library, no extra dependencies.
- **`std::vector<uint64_t>`** — the hand-rolled form. Size is $\lceil N/64 \rceil$ words; you write the indexing (`word = bits[i / 64]`, `mask = 1ULL « (i % 64)`). Use this when N is runtime-determined and you need word-level access for the bit-twiddling kernels.
- **`boost::dynamic_bitset`** — the production-grade runtime-sized bitset. Same word-array internals, plus growth / shrink, popcount, find-first / find-next, and the standard set-op overloads.

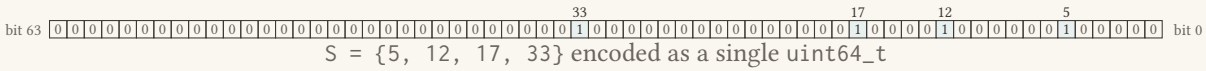
Set ops as bitwise ops

Once both sides are bitset-encoded, every binary set operation becomes a single bitwise instruction per word. Across $\lceil N/w \rceil$ words (where $w = 64$ on most CPUs):

Set op	Bitwise form
$A \cup B$	`a |= b` (per word)
$A \cap B$	`a &= b`
$A \setminus B$	`a &= ~b`
$A \triangle B$	`a ^= b`
$\lvert S \rvert$ (cardinality)	`popcount` per word, sum
contains(i)	`(a[i/64] » (i%64)) & 1`

All $O(\lceil N/w \rceil)$ word ops — linear in universe size, *not* in set cardinality, and usually 32–64× faster than the equivalent hash-set or tree-set impl due to word-level parallelism + cache streaming. Modern CPUs have a dedicated `popcnt` instruction (1 cycle latency); compilers emit it for `std::popcount` (C++20) and `__builtin_popcount`.

The picture: a 64-bit word as a bitset



`S = {5, 12, 17, 33}` encoded as a single `uint64_t`

The diagram above shows the set $\{5, 12, 17, 33\}$ over the universe $\{0, \ldots, 63\}$. Each set bit (1) is an element of S; each clear bit (0) is absent. The whole set is one `uint64_t`: `S = (1ULL « 5) | (1ULL « 12) | (1ULL « 17) | (1ULL « 33)`.

Example Word-level bit-twiddling, worked

Universe: $\{0, \ldots, 63\}$ (single 64-bit word). $A = \{5, 12, 17\}$, $B = \{12, 17, 33\}$.

- Encode: `a = (1ULL«5) | (1ULL«12) | (1ULL«17); b = (1ULL«12) | (1ULL«17) | (1ULL«33);`
- **Set bit 5**: `a |= (1ULL « 5);` (idempotent — already set; the `|=` is the right primitive even when you don't know whether the bit was already present).
- **Query bit 12**: `(a » 12) & 1ULL` returns $1 \Rightarrow 12 \in A$.
- **Cardinality**: `std::popcount(a)` returns 3; `std::popcount(b)` returns 3.
- **Union** $A \cup B$: `u = a | b;` ⇒ `popcount(u) == 4`, set is $\{5, 12, 17, 33\}$.
- **Intersection** $A \cap B$: `i = a & b;` ⇒ `popcount(i) == 2`, set is $\{12, 17\}$.
- **Difference** $A \setminus B$: `d = a & ~b;` ⇒ `popcount(d) == 1`, set is $\{5\}$.
- **Symmetric difference** $A \triangle B$: `x = a ^ b;` ⇒ `popcount(x) == 2`, set is $\{5, 33\}$.
- **Iterate the elements of** A: while `a != 0`, emit `std::countr_zero(a)` (lowest set bit), then `a &= a - 1` (clear it). Three iterations: emit 5, 12, 17.

Eight set-theoretic queries, each compiled to one or two CPU instructions. The same operations on `std::unordered_set` would cost $\sim 100\times$ more cycles per op.

Aside Forward direction: roaring bitmaps

The bitset representation breaks down when the universe is *sparse* — a 32-bit-int set with only ~ 1000 elements would be 2^{32} bits $= 512$ MB of mostly zeros. Production systems instead use *roaring bitmaps*: a 32-bit ID is split into a 16-bit "container index" + a 16-bit offset, and each container is encoded as one of three shapes (sorted-array for ≤ 4096 elements, dense bitset for ≥ 4096, run-length encoding for clustered runs). Set ops dispatch on the shape pair and stay $O(\sqrt{n})$ per op on typical data. Used in PostgreSQL's `bitmap` index scans, Apache Druid, Apache Lucene, and ClickHouse. `CRoaring` (C / C++) is the canonical implementation.

6 12.6 Disjoint-Set Union (Union-Find)

Definition —

Disjoint-Set Union (DSU), aka Union-Find. A partition of n elements into k pairwise disjoint sets. Three operations:

- `makeSet(x)`: create a new singleton $\{x\}$.
- `find(x)`: return a canonical representative ("root") of the set currently containing x. Two elements are in the same set iff `find(x) == find(y)`.
- `unite(x, y)`: replace the two sets containing x and y with their union.

The data structure as a whole is itself a Set ADT — a *set of disjoint sets*.

Use cases

- **Connectivity tracking.** "Are u and v in the same connected component?" Each `addEdge(u, v)` is a `unite`; each query is a `find` comparison.
- **Kruskal's MST (ch. 10 §10.12).** Already covered in the chapter on graphs — cycles in the partial MST are detected by "the next edge (u, v) has `find(u) == find(v)`." DSU is the algorithm-shaped reason Kruskal is $O(E \log E)$ and not $O(E \cdot V)$. Cross-link: ch. 10 §10.12 has the Kruskal application; this section has the ADT framing.
- **Tarjan's offline strongly-connected-components.** Off-line variant of SCC computation that uses DSU instead of the recursion stack from ch. 10 §10.6.
- **Online dynamic graph connectivity (incremental only).** DSU answers "are u and v connected?" under a stream of edge insertions in $O(\alpha(n))$ amortised per op. Decremental connectivity (edge *removals*) is much harder and needs different machinery.
- **Equivalence-class management.** Type-inference unification, image-segmentation flood fill, percolation theory.

Implementation: parent-pointer forest

Each set is an *up-tree*: every element points at a parent; the root points at itself and is the canonical representative. `find(x)` walks parent pointers to the root. `unite(x, y)` computes `rx = find(x)` and `ry = find(y)`, then makes one root the parent of the other.

```
struct DSU {
    std::vector<int> parent, rank_;
    explicit DSU(int n) : parent(n), rank_(n, 0) {
        for (int i = 0; i < n; ++i) parent[i] = i;
    }
    int find(int x) {
        // Path compression: every node on the find-path
        // points directly at the root after the call.
        return parent[x] == x ? x
                              : parent[x] = find(parent[x]);
    }
    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
```

```
        if (rx == ry) return false;            // already united
        // Union by rank: shorter tree under taller.
        if (rank_[rx] < rank_[ry]) std::swap(rx, ry);
        parent[ry] = rx;
        if (rank_[rx] == rank_[ry]) ++rank_[rx];
        return true;
    }
};
```

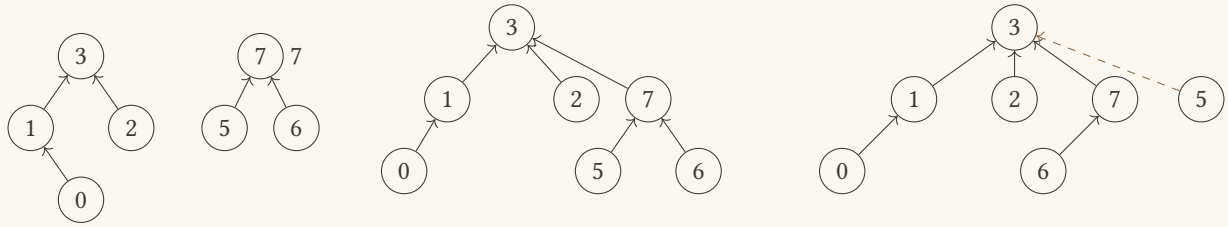
The two optimisations

- **Union by rank (or size).** Each root carries a "rank" (an upper bound on tree height; in the size-based variant, the subtree's element count instead). When uniting two trees, the shorter one is attached under the taller. Without compression this alone gives $O(\log n)$ per op (depth $\leq \log_2 n$).
- **Path compression.** During `find`, after locating the root, re-point every node on the find-path directly at the root. Flattens the tree *lazily* — the work is paid by the same query that needed the answer.

Either optimisation alone is $O(\log n)$. *Together*, the amortised cost per op drops to $O(\alpha(n))$, where α is the inverse Ackermann function — a function so slowly growing that $\alpha(n) \leq 4$ for any $n \leq 2^{2^{2^{2^{16}}}}$ (approximately 2^{65536}). For any input that fits in the universe, DSU is effectively constant per operation.

Theorem (Tarjan 1975, refined Tarjan–van Leeuwen 1984). The amortised cost of m `find` / `unite` operations on n elements with both optimisations is $\Theta(m \cdot \alpha(m, n))$. The matching lower bound shows this is optimal in the pointer-machine model. CLRS Ch 21 walks the proof; this chapter takes it as a black box.

The picture: parent forest, union step, path compression



(a) Two trees, ranks 2 and 1 (b) After `unite(0, 5)` (c) After `find(5)`: path compressed

In panel (c), the dashed arrow shows the new direct parent edge `5 -> 3` that path compression installed: previously $5 \to 7 \to 3$ (depth 2), now $5 \to 3$ (depth 1). Every subsequent `find(5)` costs one parent read instead of two.

Example 8-element DSU sequence, with path compression

Universe $\{0, 1, \ldots, 7\}$, all initially singletons. Trace the parent-pointer state through six ops:

1. Initial: `parent = [0,1,2,3,4,5,6,7]`, `rank = [0,0,0,0,0,0,0,0]`.
2. `unite(0, 1)`: $\text{find}(0) = 0$, $\text{find}(1) = 1$; ranks tied, attach $1 \to 0$ and bump `rank[0]=1`. `parent = [0,0,2,3,4,5,6,7]`.
3. `unite(2, 3)`: same shape, attach $3 \to 2$, `rank[2]=1`. `parent = [0,0,2,2,4,5,6,7]`.
4. `unite(0, 2)`: $\text{find}(0) = 0$, $\text{find}(2) = 2$; both rank-1, attach $2 \to 0$ and `rank[0]=2`. `parent = [0,0,0,2,4,5,6,7]`.
5. `unite(4, 5)`: `parent = [0,0,0,2,4,4,6,7]`, `rank[4]=1`.
6. `unite(6, 7)`: `parent = [0,0,0,2,4,4,6,6]`, `rank[6]=1`.
7. `unite(4, 6)`: both rank-1, attach $6 \to 4$, `rank[4]=2`. `parent = [0,0,0,2,4,4,4,6]`.
8. `unite(0, 7)`: $\text{find}(0) = 0$ (one hop); $\text{find}(7)$: $7 \to 6 \to 4$, returns 4. *Path compression rewrites* `parent[7]=4`. Both ranks are 2, attach $4 \to 0$, `rank[0]=3`. `parent = [0,0,0,2,0,4,4,4]`.

After step 8, *everything* is in one set rooted at 0. The `find` on 7 flattened the tree on the way through — the next `find(7)` costs one hop instead of three.

7 12.7 Bloom filter

Definition —

Bloom filter. A space-efficient probabilistic Set membership data structure with *one-sided error*:

- `contains(x) == false`: x is *definitely not* in the set. No false negatives, ever.
- `contains(x) == true`: x is *probably* in the set. False positives are possible at a tunable rate.

The structure is a single m-bit array plus k independent hash functions $h_1, \ldots, h_k : U \to \{0, \ldots, m-1\}$. Insert x by setting bits $h_1(x), \ldots, h_k(x)$; query by checking that all k bits are 1.

Construction and queries

```
#include <vector>
#include <cstdint>
#include <functional>

class BloomFilter {
    std::vector<std::uint64_t> bits;     // m bits packed in 64-bit words
    std::size_t m;                        // bit count
    int k;                                // # hash functions

    std::size_t hash(std::size_t seed, std::uint64_t x) const {
        // Cheap two-hash trick: combine std::hash with a salt.
        return (std::hash<std::uint64_t>{}(x ^ seed)) % m;
    }
public:
    BloomFilter(std::size_t m_, int k_)
        : bits((m_ + 63) / 64, 0), m(m_), k(k_) {}

    void insert(std::uint64_t x) {
```

```
        for (int i = 0; i < k; ++i) {
            std::size_t b = hash(i * 0x9E3779B97F4A7C15ULL, x);
            bits[b / 64] |= (1ULL << (b % 64));
        }
    }
    bool contains(std::uint64_t x) const {
        for (int i = 0; i < k; ++i) {
            std::size_t b = hash(i * 0x9E3779B97F4A7C15ULL, x);
            if (!(bits[b / 64] & (1ULL << (b % 64)))) return false;
        }
        return true;            // probably present (or false positive)
    }
};
```

No `erase`. A naive Bloom filter cannot delete: any bit set might be "shared" between multiple inserted keys, so clearing it could turn a true positive into a false negative, breaking the one-sided-error contract. The counting-Bloom variant in the next notebox restores delete by replacing each bit with a small counter.

The false-positive-rate formula

After inserting n keys into an m-bit table with k hash functions (assumed independent and uniform), the probability that a specific bit is still 0 is $(1 - 1/m)^{kn} \approx e^{-kn/m}$. A query for a never-inserted y is a false positive iff all k of its bits happen to be set, which happens with probability:

$$p \;\approx\; \left(1 - e^{-kn/m}\right)^k.$$

Differentiating w.r.t. k and setting $\frac{\partial p}{\partial k} = 0$ gives the optimal number of hash functions:

$$k^\star \;=\; \frac{m}{n} \ln 2 \;\approx\; 0.6931 \cdot \frac{m}{n}.$$

At $k = k^\star$, the false-positive rate simplifies to $p^\star = (1/2)^{k^\star}$, equivalently $m \approx -n \log_2 p^\star / \ln 2 \approx 1.44 \cdot n \cdot \log_2(1/p^\star)$ bits. Useful calibrations:

$p^\star$	m/n	$k^\star$
0.10	4.79	4
0.01	9.59	7
0.001	14.38	10
10^{-4}	19.18	14
10^{-5}	23.97	17

A 1% false-positive rate costs ~ 9.6 bits per element. Compare `std::unordered_set`, which costs ~ 32 bytes per element ($256\times$ more). That space ratio is the entire business case for Bloom filters.

Use cases

- **Database existence pre-check.** "Is this key *maybe* in the table?" Avoid a disk read when the Bloom filter says no. RocksDB and LevelDB attach a per-SST-file Bloom filter so LSM compaction does not chase missing keys through every level.
- **CDN edge caching.** "Is this URL *maybe* in the cache?" False-positive cost: one wasted RAM probe. False-negative cost: a cache miss that should have been a hit. The asymmetry justifies one-sided error.

- **Network packet filtering.** Squid web cache, firewall blacklist pre-checks. Cheap reject of known non-members before the expensive deep inspection.
- **DNS negative caching.** "Have we recently failed to resolve this name?" Bloom filter answers in microseconds; falling back on DNS itself takes milliseconds.
- **Distributed-systems set reconciliation.** Compare two large sets between nodes by exchanging Bloom filters first — only fall back on full set reconciliation if the filters say there might be a difference.

Aside Variants beyond the textbook Bloom filter

- **Counting Bloom filter.** Replace each bit with a c-bit counter (typically $c = 4$). `insert` increments; `erase` decrements. False-positive analysis is unchanged but now you can *remove*. $4\times$ space cost vs. a plain Bloom filter.
- **Cuckoo filter.** Stores small *fingerprints* (8–16 bits) in a cuckoo-hash table; supports delete and gives a strictly better false-positive rate at the same space than counting Bloom. Used in CockroachDB.
- **Quotient filter / Xor filter.** Newer designs with cache-friendlier layouts; the Xor filter (Graf and Lemire, 2020) is read-only and matches Bloom space at $\sim 30\%$ better lookup throughput.
- **Scalable Bloom filter.** Chained Bloom filters that grow on demand — maintains a target FPR even when n is unknown a priori.

The original paper is Burton Bloom (1970); the canonical modern survey is Broder and Mitzenmacher (*Internet Mathematics*, 2004), enumerated in §12.8.

8 12.8 Production references and further reading

Aside Where Set impls actually live in production

C++ STL.

- `std::set` / `std::map` — red-black tree (cross-link **ch. 9**). Ordered iteration, `lower_bound` / `upper_bound`, $O(\log n)$ worst case on all three core ops. The default "ordered Set" impl.
- `std::unordered_set` / `std::unordered_map` — open-addressing or chained hash table (cross-link **ch. 5**). $O(1)$ expected on all three core ops; no order; the default "unordered Set" impl.
- `std::multiset` / `std::multimap` — red-black tree with duplicate keys allowed. The three behavioural deltas in §12.4 apply.
- `std::bitset<N>` — compile-time-fixed bitset. `boost::dynamic_bitset` is the runtime-sized counterpart.

Higher-performance hash sets.

- `absl::flat_hash_set` (Google Abseil) — Swiss Tables open-addressing hash with SIMD-vectorised probing. ~ 2–$3\times$ faster than `std::unordered_set` on most workloads. Used inside Chrome, TensorFlow, Envoy.
- `phmap::flat_hash_set` (parallel-hashmap) — drop-in replacement with sharded internal locking for concurrent use.
- `folly::F14ValueSet` (Facebook Folly) — 14-way probe SIMD hash with a different layout / cache trade-off.

Beyond C++.

- Python `set` / `frozenset` — open- addressing hash; `frozenset` is hashable.
- Java `HashSet` / `TreeSet` — chained hash / red-black; Java 9+ `Set.of(...)` is the immutable form.
- Rust `HashSet` / `BTreeSet` — hash (SipHash by default for HashDoS resistance) / B-tree-of-arrays (cross-link **ch. 11**).

Specialised set structures.

- `CRoaring` — compressed bitmap library (cross-link §12.5 forward-direction notebox). Used in Apache Druid, Lucene, ClickHouse, PostgreSQL bitmap scans.
- `libcuckoo` — concurrent cuckoo hash set.
- RocksDB / LevelDB Bloom filter implementations — one Bloom per SST file in the LSM tree.

Canonical references.

- *CLRS* Ch 11 (hash tables), Ch 12 (BSTs as sets), Ch 13 (red-black trees), Ch 21 (disjoint-set structure).
- Broder and Mitzenmacher, *"Network applications of Bloom filters: A survey,"* Internet Mathematics, 2004.
- Tarjan, *"Efficiency of a good but not linear set union algorithm,"* JACM 22(2), 1975 — the $\alpha(n)$ bound for DSU.
- Fredman, Komlós, Szemerédi, *"Storing a sparse table with $O(1)$ worst case access time,"* JACM 31(3), 1984 — the FKS perfect-hash construction.
- Bloom, *"Space/time trade-offs in hash coding with allowable errors,"* CACM 13(7), 1970 — the original Bloom-filter paper.
- Mitzenmacher and Upfal, *Probability and Computing* — standard text for the probabilistic analyses underpinning Bloom filters and randomised data structures.

Key idea

Chapter takeaway. Sets are the deduplicating, order-free, membership-query-oriented sibling of lists. The chapter unifies under the impl-decision matrix — one ADT, seven impls, each tuned to a different axis of the trade-off:

- **Hash set** (**ch. 5**, `std::unordered_set`) — $O(1)$ expected, no order. The default unless something else is needed.
- **Balanced BST** (**ch. 9**, `std::set`) — $O(\log n)$ worst case, ordered iteration, `lower_bound`.
- **B-tree / B+ tree** (**ch. 11**, SQLite / PostgreSQL / MySQL InnoDB) — $O(\log_K n)$, the on-disk version of the same operations.
- **Multiset** (§12.4) — duplicates allowed; one comparator relaxation.
- **Bitset** (§12.5) — small dense universe; $O(1)$ ops, 32–64$\times$ faster constant than hash.
- **Disjoint-Set Union** (§12.6) — partition ADT; $O(\alpha(n))$ amortised; powers Kruskal (**ch. 10** §10.12).
- **Bloom filter** (§12.7) — probabilistic, one- sided error; ~ 10 bits / element at 1% FPR.
- **Perfect hash** (§12.3) — static set, $O(1)$ *worst* case; `gperf`, `phf`.

The recurring move: *iterate one side, probe the other*, with the probe shape (sorted-merge / hash-probe / bit-and) determined by the impl pair. The ADT is simple; the engineering choices it unlocks are not.
Looking ahead. Past the cs-300 boundary the Set ADT extends in three directions. *Concurrent / lock-free sets*: Cliff Click's non-blocking hash map, Java's `ConcurrentSkipListSet`, RCU-protected read-mostly sets, and the lock-free hash-table research line (Michael 2002, Shavit 2008). *Sparse-bitmap*

research: roaring bitmaps (§12.5 forward-direction notebox), tree-of-bitmaps, succinct data structures with $H_0(n) + o(n)$ bits. *Set theory proper*: powerset lattices (the partial-order structure on $\mathcal{P}(X)$), Boolean algebras, partial orders, the connection from finite-set ADTs to the algebraic structures that underpin databases (relational algebra is set algebra) and type theory (subtyping is subset-of-values). cs-300 stops at the ADT layer; the references in §12.8 are the entry points for continuing past it.